

lab3

要求：实现 `brk` `mmap` `munmap` 测试样例的要求

▼ 实验前置：

- 修改 `init.c`，添加这几个测试样例点

```
char *tests[] = {
    // "getcwd",
    // "write",
    // "getpid",
    // "times",
    // "uname",

    // // lab2

    // "wait",
    // "clone",
    // "fork",
    // "execve",
    // "getppid",
    // "exit",
    // "yield",
    // "waitpid",
    // "gettimeofday",
    // "sleep",

    // // lab3

    "brk",
    "mmap",
    "munmap",
};
```

- 执行 `make fast_renew`，看看结果

```

init: starting brk
===== START test_brk =====
pid 2 brk: unknown sys call 214
Before alloc,heap pos: -1
pid 2 brk: unknown sys call 214
pid 2 brk: unknown sys call 214
After alloc,heap pos: -1
pid 2 brk: unknown sys call 214
pid 2 brk: unknown sys call 214
Alloc again,heap pos: -1
===== END test_brk =====
init: starting mmap
===== START test_mmap =====
file len: 0
pid 3 mmap: unknown sys call 222
mmap error.
===== END test_mmap =====
init: starting munmap
===== START test_munmap =====
file len: 0
pid 4 munmap: unknown sys call 222
mmap error.
===== END test_munmap =====

```

查询https://github.com/oscomp/testsuite-for-oskernel/blob/main/oscomp_syscalls.md这个文档，我们可以找到这几个调用号对应的定义：

- 214: `#define SYS_brk 214`
- 222: `#define SYS_mmap 222`
- 进行一系列注册过程，需要修改的脚本有： `sysnum.h` `syscall.c` `usys.pl`
- 接下来为了能通过编译，我们需要写几个占位用的函数
 - 进入 `sysproc.c`，挨个搜索函数，我们发现有一个系统实现的 `sys_sbrk()`，于是我们自己建一个 `sys_brk()`，先直接调用这个实现好的函数

```

uint64
sys_brk(void)

```

```
{
    return sys_sbrk();
}
```

- 对于mmap，没有发现已实现好的函数，我们就写一个这个应付一下

```
uint64
sys_mmap(void)
{
    return 0;
}
```

- 然后我们继续 `make fast_renew`，结果是这样

```
init: starting brk
===== START test_brk =====
Before alloc,heap pos: 20480
After alloc,heap pos: 41024
Alloc again,heap pos: 82112
===== END test_brk =====
init: starting mmap
===== START test_mmap =====
file len: 0
mmap content: (null)
pid 3 mmap: unknown sys call 215
===== END test_mmap =====
init: starting munmap
===== START test_munmap =====
file len: 0
pid 4 munmap: unknown sys call 215
munmap return: -1
-- Assert Fatal ! ---
```

- 我们检查一下，215调用号对应的是 `#define SYS_munmap 215`，这样我们仍然用占位置的函数注册 `sys_munmap()` 以后，再运行就会得到这样的结果，接下来就可以挨个实现这些调用了

```

init: starting brk
===== START test_brk =====
Before alloc,heap pos: 20480
After alloc,heap pos: 41024
Alloc again,heap pos: 82112
===== END test_brk =====
init: starting mmap
===== START test_mmap =====
file len: 0
mmap content: (null)
===== END test_mmap =====
init: starting munmap
===== START test_munmap =====
file len: 0
munmap return: 0
munmap successfully!
===== END test_munmap =====

```

▼ brk:

- 检查测试样例:

```

/*
 * 测试通过时应输出:
 * "Before alloc,heap pos: [num]"
 * "After alloc,heap pos: [num+64]"
 * "Alloc again,heap pos: [num+128]"
 *
 * Linux 中brk(0)只返回0, 此处与Linux表现不同, 应特殊说明.
 */
void test_brk(){
    TEST_START(__func__);
    intptr_t cur_pos, alloc_pos, alloc_pos_1;

    cur_pos = brk(0);
    printf("Before alloc,heap pos: %d\n", cur_pos);
    brk(cur_pos + 64);
    alloc_pos = brk(0);

```

```

    printf("After alloc,heap pos: %d\n",alloc_pos);
    brk(alloc_pos + 64);
    alloc_pos_1 = brk(0);
    printf("Alloc again,heap pos: %d\n",alloc_pos_1);
    TEST_END(__func__);
}

```

- 这里的brk(n)会调用用户态的

```

int brk(void *addr)
{
    return syscall(SYS_brk, addr);
}

```

而这里的传参n被包装成一个addr形式，而观察测试样例的要求，三次输出的结果应该是逐次递增64的。

- 分析一下目前直接调用的sys_sbrk():

```

uint64
sys_sbrk(void)
{
    int addr;
    int n;

    if (argint(0, &n) < 0)
        return -1;
    addr = myproc()->sz;
    if (growproc(n) < 0)
        return -1;
    return addr;
}

```

其中，`myproc()` 是一个定义在 `proc.c` 下的 `struct proc *` 指针，它的作用是返回当前cpu正运行的进程地址，这个时候我们去看一下 `struct proc` 的结构，每一个成员都是什么含义：

```

// Per-process state
struct proc

```

```

{
    struct spinlock lock;

    // p->lock must be held when using these:
    enum procstate state; // Process state
    struct proc *parent;  // Parent process
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    int xstate;           // Exit status to be returned to parent's wait
    int pid;              // Process ID

    // these are private to the process, so p->lock need not be held.
    uint64 kstack;        // Virtual address of kernel stack
    uint64 sz;            // Size of process memory (bytes)
    pagetable_t pagetable; // User page table
    pagetable_t kpagetable; // Kernel page table
    struct trapframe *trapframe; // data page for trapframe.S
    struct context context;      // switch() here to run process
    struct file *ofile[NOFILE]; // Open files
    struct dirent *cwd;          // Current directory
    char name[16];              // Process name (debugging)
    int tmask;                  // trace mask
};

```

需要持有 `p->lock` 才能访问的成员：

- `enum procstate state`：进程当前的状态（例如：`UNUSED` 未使用、`SLEEPING` 睡眠/等待中、`RUNNABLE` 就绪可运行、`RUNNING` 正在运行、

`ZOMBIE` 僵尸状态)。

- `struct proc *parent`：指向当前进程的父进程的指针。用于在子进程退出时通知父进程。
- `void *chan`：通道标识 (channel)。如果非 0，表示当前进程正在睡眠，且正在等待某个特定的事件 (通常是某个内核数据结构的内存地址，比如锁的地址、磁盘缓冲区的地址等)。
- `int killed`：进程是否已被杀死的标志位。如果非 0，表示有其他进程请求终止该进程 (如通过 `kill` 系统调用)，它会在下一次陷入内核或从内核返回用户态前退出。
- `int xstate`：退出状态码 (Exit status)。当进程调用 `exit(status)` 退出成为僵尸进程 (ZOMBIE) 后，其退出码存放在此，等待父进程通过 `wait()` 收集。
- `int pid`：进程的全局唯一标识符 (Process ID)。

进程的私有成员，不需要持有锁：

- `uint64 kstack`：进程的内核栈 (Kernel stack) 的虚拟地址。当进程从用户态陷入内核态运行系统调用或处理中断时，使用的就是这个栈。
- `uint64 sz`：进程当前占用的用户内存大小，以字节 (bytes) 为单位。当使用 `sys_sbrk()` 等扩展内存时，这个值会改变。
- `pagetable_t pagetable`：用户态页表指针。由于每个进程有自己独立的用户虚拟地址空间，这个页表记录了该进程的用户虚拟地址到物理内存地址的映射。
- `pagetable_t kpagetable`：内核态页表指针。这个项目 (可能修改自标准的 xv6) 中，每个进程都拥有独立的内核页表结构，以便在内核态也维持精细的内存隔离或映射层级。
- `struct trapframe *trapframe`：陷入帧 (Trapframe)。一个指向专门的数据页的指针，用于在用户态发生系统调用或中断陷入内核态 (如 `trampoline.S` 中执行操作) 时，保存用户态的寄存器状态，以便之后可以恢复执行。
- `struct context context`：进程上下文。用于内核线程 (进程) 的切换。当在内核中调用 `swtch()` 函数挂起进程时，会将 CPU 的被调用者保存寄存器 (如 `s0-s11, ra, sp` 等) 保存在这里。

- `struct file *ofile[NOFILE]`：文件描述符表。这是一个指针数组，记录了进程当前打开的所有文件资源。数组索引就是文件描述符（fd）。
 - `struct dirent *cwd`：当前工作目录（Current working directory）。这代表了进程所处的目录节点。
 - `char name[16]`：进程名称。系统通常允许给进程指定一个简短的名字，主要方便调试代码或通过 `ps` 等工具查看进程情况。
 - `int tmask`：系统调用追踪掩码（Trace mask）。用于实现类似 `strace` 的功能，拦截和输出特定的系统调用，调试工具会用到。
- 综合两部分的内容，我们其实分析出来：`sys_sbrk()` 的主要作用是将用户态传的参数 `n` 解码为扩张该运行中进程内存的大小 `n` 个Byte，如果成功扩张内存，则返回扩张内存之前的进程内存大小。
- 这里其实还有一个很关键的信息，就是如果我们深入了解一下xv6系统的内存分配方式，我们可以发现所有进程的虚拟空间都是从0开始向上增长的，这个信息可以查询 `Makefile` 或者我们直接从 `sys_sbrk()` 中，可以推测出。
 - 由于 `addr = myproc() -> sz`，我们可以发现，这里的 `sz` 实际上被解码为一个地址。因此我们得到一个关键信息，就是在xv6的进程内存分配中，所有虚拟地址都是从0开始向上增长，而管理进程用的结构体 `struct proc`，其中的 `sz` 变量，实际上也是整个进程空间的堆顶地址。
- 观察目前我们直接调用 `sys_sbrk()` 的结果：

```
Before alloc,heap pos: 20480
After alloc,heap pos: 41024
Alloc again,heap pos: 82112
```

其实就可以发现 $41024 = 20480 + 20480 + 64$ ，也就是说，如果我们直接使用 `sbrk` 的逻辑，那么我们的 `brk` 测试样例实际上做了这样一件事：

```
cur_pos = brk(0);    // 分配了0个Byte，并返回了当前的堆顶地址
printf("Before alloc,heap pos: %d\n", cur_pos);
brk(cur_pos + 64);   // 又分配了堆顶地址+64大小的Byte
```



```

    alloc_pos = brk(0); // 分配0个Byte, 这一步实际上
    就是获取当前堆顶地址
    printf("After alloc,heap pos: %d\n",alloc_pos);
    brk(alloc_pos + 64); // 又分配了堆顶地址+64大小的Byte
    alloc_pos_1 = brk(0);
    printf("Alloc again,heap pos: %d\n",alloc_pos_1);

```

因此返回的内容完全不是相差64。

- 既然已经知道了问题所在，我们只需要修改一点 `sbrk` 的逻辑即可满足 `brk` 的要求，在这里，`brk` 传入的参数不同于 `sbrk`，后者传入的是需要扩展的Byte数，而前者则是直接传入一个希望扩展到的地址：

```

uint64
sys_brk(void)
{
    int addr;
    int endaddr;

    if (argint(0, &endaddr) < 0)
        return -1;
    if (endaddr == 0)
        return myproc()->sz;
    addr = myproc()->sz;
    if (growproc(endaddr - addr) < 0)
        return -1;
    return endaddr;
}

```

这里要注意：正如助教在测试样例中写的——Linux 中 `brk(0)` 只返回0，此处与Linux表现不同，应特殊说明。——这里的 `brk(0)` 事实上是要返回当前堆顶地址。

▼ mmap:

- 先查看测试样例：

```

/*
 * 测试成功时输出:
 * " Hello, mmap success"
 * 测试失败时输出:
 * "mmap error."
 */
static struct kstat kst;
void test_mmap(void){
    TEST_START(__func__);
    char *array;
    const char *str = " Hello, mmap successfully!";
    int fd;

    fd = open("test_mmap.txt", O_RDWR | O_CREATE);
    write(fd, str, strlen(str));
    fstat(fd, &kst);
    printf("file len: %d\n", kst.st_size);
    array = mmap(NULL, kst.st_size, PROT_WRITE | PROT
_READ, MAP_FILE | MAP_SHARED, fd, 0);
    //printf("return array: %x\n", array);

    if (array == MAP_FAILED) {
        printf("mmap error.\n");
    }else{
        printf("mmap content: %s\n", array);
        //printf("%s\n", str);

        munmap(array, kst.st_size);
    }

    close(fd);

    TEST_END(__func__);
}

```

接下来我们挨个看这个测试样例的调用情况。

▼ open:

- 检查 `open` 的调用号，我们发现 `sys_open` 事实上是文档中的 `#define SYS_openat 56`，而我们再调查一下，发现事实上我们的测试中是有 `open` 和 `openat` 这两个测试点的！所以我们还是一个一个实现一下。
- 检查 `open.c` 的测试要求：

```
void test_open() {
    TEST_START(__func__);
    // O_RDONLY = 0, O_WRONLY = 1
    int fd = open("./text.txt", 0);
    assert(fd >= 0);
    char buf[256];
    int size = read(fd, buf, 256);
    if (size < 0) {
        size = 0;
    }
    write(STDOUT, buf, size);
    close(fd);
    TEST_END(__func__);
}
```

要求就是打开一个文件名，然后 `read` 出这个文件中的内容，如果 `open` 成功，会返回给我们一个文件描述符 `fd`，以后我们就可以用 `fd` 来访问这个文件了，随后，我们用这个 `fd` 将读出来的东西写进标准输出 `STDOUT` 中即可。

- 然后我们观察一下传参情况：

```
int open(const char *path, int flags)
{
    return syscall(SYS_openat, AT_FDCWD, path,
        flags, 0_RDWR);
}
```

这里的系统调用号实际上还是 `openat` 的调用号，而我们又可以在 `sysfile.c` 里直接找到 `sys_open` 因此我们只需要在内核态继续完善 `sys_open` 就行了，传给内核态 `sys_open` 的第一个参数是 `AT_FDCWD`，经过查询，我们得知它的值是 -100，它在这个参数的含义是：请帮我打开这个 `path` 文件。如果这是一个相对路径的话，就在当前进程的工作目录下找它（因为我传的是 -100 即 `AT_FDCWD`），这个参数的全称其

实是"**At File Descriptor Current Working Directory**"。这里的后两个参数我们参考文档，也解释一下，这个会帮助我们在后面完成 `openat` 的全流程：

- `flags`（打开方式标志）
它告诉内核你当前操作文件的意图。通常由多个宏通过按位或（`|`）组合而成：

读写意图：`O_RDONLY`（只读），`O_WRONLY`（只写），`O_RDWR`（读写）。这三个必须选且只能选其一。
附加行为：
`O_CREAT`：文件如果不存在，请帮我创建它。
`O_TRUNC`：如果文件已存在，打开时顺便把里面清空（截断长度设为 0）。
`O_APPEND`：追加模式，每次写入都会强行写到文件末尾。
- 最后一个 `mode`（文件创建权限）表示文件的读写执行权限，只有在 `flags` 中包含了 `O_CREAT` 时才是有效且必须的，而这个位置的参数通常是一个八进制数（比如 `0600`，`0666`，`0777`）。

- 接下来我们开始研究原有的 `sys_open()`

```
uint64
sys_open(void)
{
    char path[FAT32_MAX_PATH];
    int fd, omode;
    struct file *f;
    struct dirent *ep;

    if (argstr(0, path, FAT32_MAX_PATH) < 0 || argint(1, &omode) < 0)
        return -1;

    // 要在物理磁盘上新建一个文件。
    // 后面有关ep的这部分其实都是在处理创建文件的事，我们就不看了。

    if (omode & O_CREATE)
    {
        ep = create(path, T_FILE, omode);
```

```

    if (ep == NULL)
    {
        return -1;
    }
}
else
{
    if ((ep = ename(path)) == NULL)    // 这一步是
    查找数据盘上该路径是否已有文件占用
    {
        return -1;
    }
    eunlock(ep);
    if ((ep->attribute & ATTR_DIRECTORY) && omode
    != O_RDONLY)
    {
        eunlock(ep);
        eput(ep);
        return -1;
    }
}

```

// 在一个全局系统文件表中为f分配一个file结构体，即一个打开文件句柄

// 括号中的左边如果正常执行，则左侧判断为0，右侧开始给fd分配一个文件描述符

```

    if ((f = filealloc()) == NULL || (fd = fdalloc
    (f)) < 0)
    {
        if (f)
        {
            fileclose(f);
        }
        eunlock(ep);
        eput(ep);
        return -1;
    }

```

```

// trunc相关逻辑，也不用动

    if (!(ep->attribute & ATTR_DIRECTORY) && (omode
& O_TRUNC))
    {
        etrunc(ep);
    }

// 调整f句柄的一些状态量，如权限等

    f->type = FD_ENTRY;
    f->off = (omode & O_APPEND) ? ep->file_size : 0;
    f->ep = ep;
    f->readable = !(omode & O_WRONLY);
    f->writable = (omode & O_WRONLY) || (omode & O_R
DWR);

    eunlock(ep);
// 最终返回一个文件描述符
    return fd;
}

```

- `struct file *f;`
`struct dirent *ep;`：这两个要放在一起考虑，经过查询定义，我们知道下面的 `struct dirent` 实际上代表着在物理磁盘上，这个文件的真实情况，也就是说操作系统的全局中，它的信息只对应一个文件，这是唯一的，又称为这个文件的元数据。
而 `struct file` 相当于一个句柄，关联的是进程对一个文件的打开状态和当前读写权限，这个并不唯一，如果进一步看，这里的 `f->ep`，倘若两个进程都打开了“test.txt”，则会指向同一个 `dirent` 实例。
- 这块分配 `fd` 时，会在一个 `proc.h`（进程的定义头文件）中定义好的进程打开文件表 `struct file *ofile[NOFILE];` 中查找未被占用的条目，并将这个 `f` 安置在这个位置，并返回当前这个位置的索引值作为 `fd`。
- 所以我们其实首先做的第一个事，就是改变一下内核态读取参数的位置，因为在用户态传参中，第一个参数传的是 `AT_FDCWD` 而非 `path`。此外，我们发现这里的 `omode` 实际上就是我们在用户态传的 `flags`，`omode` 就是 `open`

mode，总之我们要明白这个 `omode` 其实就是用户态指定的文件打开权限。

```
if (argstr(1, path, FAT32_MAX_PATH) < 0 || argint
    (2, &omode) < 0)
    return -1;
```

只需要这样小小修改，就可以通过open的测试点啦！其实还有意外之喜，如果我们留心观察一下open.c的测试样例，我们可以发现它还调用了read，再看一眼测试样例中，确实有read.c，而这个测试样例实际上又依赖open的实现，我们什么都不用改，直接把read也放在init.c的测试样例中，read就可以一并通过了。

▼ openat:

- 看一下这个脚本：

```
void test_openat(void) {
    TEST_START(__func__);
    //int fd_dir = open(".", O_RDONLY | O_CREATE);
    int fd_dir = open("./mnt", O_DIRECTORY);
    printf("open dir fd: %d\n", fd_dir);
    int fd = openat(fd_dir, "test_openat.txt", O_C
REATE | O_RDWR);
    printf("openat fd: %d\n", fd);
    assert(fd > 0);
    printf("openat success.\n");
    close(fd);

    TEST_END(__func__);
}
```

观察一下，我们发现在第一次调用 `open` 时，传入的第二个参数 `flags` 是一个之前我们没见过的 `O_DIRECTORY`，这会导致我们在测试时得到这样的报错

```
===== START test_openat =====
open dir fd: -1
```

```
openat fd: -1
-- Assert Fatal ! ---
```

这是因为我们当前的 `open` 逻辑实际上是这样的，就是说如果这个 `ep` 在数据盘中只作为一个目录，并且传来的 `flags` (`omode`) 不是单纯的只读模式 (`O_RDONLY`)，就返回-1，相当于一个错误。

```
if ((ep->attribute & ATTR_DIRECTORY) && omode != O_RDONLY)
{
    eunlock(ep);
    eput(ep);
    return -1;
}
```

这就是导致测试时得到-1的原因，我们打开了一个目录，但是却传入了内核态没见过的 `O_DIRECTORY` 参数。

- 我们在用户态查找所有有关打开方式的参数

```
#define O_RDONLY 0x000
#define O_WRONLY 0x001
#define O_RDWR 0x002 // 可读可写
// #define O_CREATE 0x200
#define O_CREATE 0x40
#define O_DIRECTORY 0x0200000

#define DIR 0x040000
#define FILE 0x100000

#define AT_FDCWD -100
```

并对比在内核态下的定义：

```
#define O_RDONLY 0x000
#define O_WRONLY 0x001
#define O_RDWR 0x002
#define O_APPEND 0x004
```



```
#define O_CREATE 0x200
#define O_TRUNC 0x400
```

我们让内核态的定义都和用户态对其才行，于是修改后得到这样

```
#define O_RDONLY 0x000
#define O_WRONLY 0x001
#define O_RDWR 0x002
#define O_APPEND 0x004
#define O_CREATE 0x40 // 修改，原本为0x200
#define O_TRUNC 0x400

#define O_DIRECTORY 0x0200000
#define AT_FDCWD -100
```

- 做完这些以后，我们再修改一下刚才会产生报错的部分，让它兼容性更高，可以返回一个目录的文件描述符

```
else
{
    if ((ep = ename(path)) == NULL)
    {
        return -1;
    }
    elock(ep);

    // 【修改点 1】：如果参数要求必须是目录，但找到的实体不是目录，返回-1
    if ((omode & O_DIRECTORY) && !(ep->attribute & ATTR_DIRECTORY))
    {
        eunlock(ep);
        eput(ep);
        return -1;
    }

    // 【修改点 2】：放宽限制，只要没有要求写权限 (O_WRONLY / O_RDWR)，不强求 omode == 0，这让宏标签共存成为可能
```

```

        if ((ep->attribute & ATTR_DIRECTORY) && ((omode & 3) != 0_RDONLY))
        {
            eunlock(ep);
            eput(ep);
            return -1;
        }
    }
}

```

做到这一步我们再运行测试样例，就可以通过了。

- 但事实上，如果只是这样写的话，就完全没有考虑到之前提到的参数 `AT_FDCWD` 和 `dirfd`，而这两个参数才是我们实现的功能更强的 `openat` 的关键：即如果传入的第一个参数是代表着“**At File Descriptor Current Working Directory**”即在当前进程运行的文件目录下查找 `path` 指定的文件并打开，而如果第一个参数传进去的并非 `AT_FDCWD`，则查找 `dirfd` 下的 `path` 进行打开。

```

int open(const char *path, int flags)
{
    return syscall(SYS_openat, AT_FDCWD, path, flags, 0_RDWR);
}

int openat(int dirfd, const char *path, int flags)
{
    return syscall(SYS_openat, dirfd, path, flags, 0600);
}

```

因此我们接下来将这个 `dirfd` 参数加入函数逻辑中。

- 这时候我们仍然要把在 `brk` 中看过的 `proc` 结构体拿出来，分析一下我们该做什么。

```

// Per-process state
struct proc
{
    ...
}

```

```

    // these are private to the process, so p->lock
    need not be held.
    uint64 kstack;                // Virtual address
    ss of kernel stack
    uint64 sz;                    // Size of process
    ss memory (bytes)
    pagetable_t pagetable;        // User page table
    pagetable_t kpagetable;       // Kernel page table
    struct trapframe *trapframe; // data page for
    trampoline.S
    struct context context;        // swtch() here
    to run process
    struct file *ofile[NOFIL];    // Open files
    struct dirent *cwd;            // Current directory
    char name[16];                // Process name
    (debugging)
    int tmask;                    // trace mask
};

```

在这里，我们注意到有这样的 `dirent` 实例 `cwd`，它对应着目前这个进程正在运行的物理位置，即这个文件或目录在内存中缓存的物理元数据（类似Linux中的inode，在我们ics中讲到过这个）。

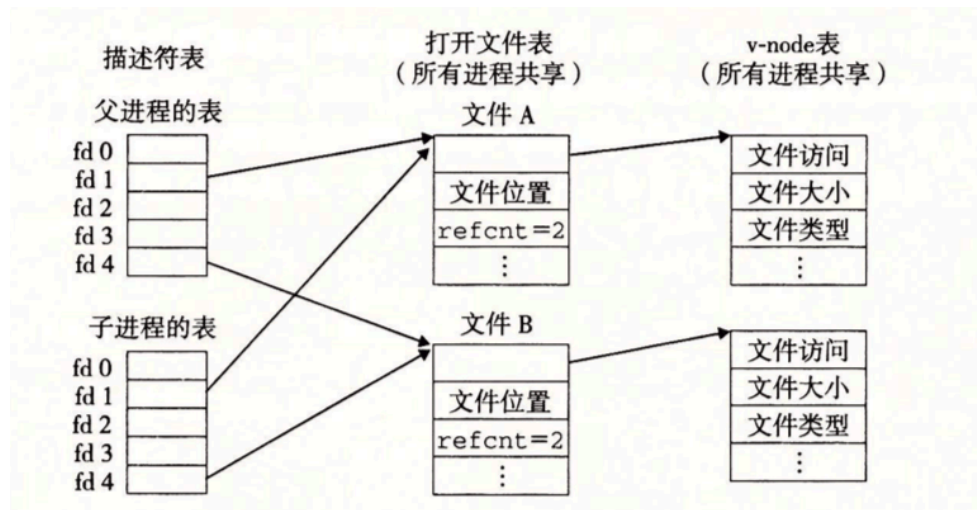


图 10-14 子进程如何继承父进程的打开文件。初始状态如图 10-12 所示

可以认为dirent就是vnode表的条目，file就是打开文件表的条目，fd就是描述符表的条目

总之，深一步理解 `struct dirent`，它就是一个目录结构体，这个结构体直接对应着真实内存的位置，而之前的 `struct file`，则是它的一个句柄，用以记录对应的 `dirent` 的真实文件在当前打开时的状态（如读写等）。

这里的 `file*` 数组，则是该进程打开文件的对应表，一个进程最多只能同时打开 `NOFILE` 个文件，也就是说最多情况下，每个条目的 `file*` 都会指向一个全局进程共用的 `file` 实例，并对应一个真实 `dirent`。

- 而在用户态，我们第一次调用 `int fd_dir = open("./mnt", O_DIRECTORY);` 时，已经将需要查找文件名的路径 `./mnt` 加载到了该进程中，如果观察返回结果，我们直到在该进程中，`./mnt` 目录的 `fd=3`（即 `stdin`、`stdout`、`stderr` 之后的第一个文件描述符）。

也就是说在这个进程的 `struct proc` 下的 `ofile[]` 中，已经分配好了一个条目，这个条目指向的是全局中 `./mnt` 的 `file` 实例，并对应着物理空间中的一个 `dirent`。这一步，是依靠这一个调用实现的

```
if ((f = filealloc()) == NULL || (fd = fdalloc(f)) < 0)
```

于是我们就可以通过 `myproc()->ofile[fd]` 这样的办法，取到 `./mnt` 这个目录对应的 `*file` 了

- 接下来，我们先 `printf` 一下，看看是不是和分析的一样：

```

uint64
sys_open(void)
{
    char path[FAT32_MAX_PATH];
    int fd, omode, dirfd;
    struct file *f;
    struct dirent *ep;

    if (argint(0, &dirfd) < 0 || argstr(1, path,
    FAT32_MAX_PATH) < 0 || argint(2, &omode) < 0)
        return -1;

    printf("sys_open: dirfd=%d, path=%s, omode=%d
    \n", dirfd, path, omode);
    ...
}

```

结果是这样，但其实fd=4对应的到底是什么，我们并不知道它是否为一个合法的文件。

```

sys_open: dirfd=-100, path=./mnt, omode=2097152
open dir fd: 3
sys_open: dirfd=3, path=test_openat.txt, omode=
66
openat fd: 4
openat success.

```

- 在原先的逻辑中，我们关注这一句，这里的 `ename` 事实上就是整个 `open` 的寻址函数关键

```

if ((ep = ename(path)) == NULL)

```

检查定义，注意到了这两个函数：

```

static struct dirent *lookup_path(char *path, i
nt parent, char *name)
{
    struct dirent *entry, *next;

```

```

// 检查是否为绝对路径
if (*path == '/') {
    entry = edup(&root);
} else if (*path != '\0') {
    entry = edup(myproc()->cwd); // 默认从
进程当前运行中的目录开始查找
} else {
    return NULL;
}
while ((path = skipelem(path, name)) != 0)
{
    elock(entry);
    if (!(entry->attribute & ATTR_DIRECTOR
Y)) {
        eunlock(entry);
        eput(entry);
        return NULL;
    }
    if (parent && *path == '\0') {
        eunlock(entry);
        return entry;
    }
    if ((next = dirlookup(entry, name, 0))
== 0) {
        eunlock(entry);
        eput(entry);
        return NULL;
    }
    eunlock(entry);
    eput(entry);
    entry = next;
}
if (parent) {
    eput(entry);
    return NULL;
}
return entry;
}

```

```

struct dirent *ename(char *path)
{
    char name[FAT32_MAX_FILENAME + 1];
    return lookup_path(path, 0, name);
}

```

这两个函数的作用，其实是把一串给人看的路径

(如"./mnt/user/src")，沿着目录树找下去，最终转化为系统操作文件系统所需要的内核数据结构 `struct dirent *`，而我们看到

`lookup_path` 的默认行为是先检查路径开头是否为"/"，即是否为一个从根目录开始的绝对路径，如果不是，则从 `myproc` 的运行路径开始查找相对路径 `path` 对应的 `dirent`。

- 所以在调用 `ename(path)` 之前，我们就要对 `path` 进行逻辑上的处理了，如果在调用 `open()` 时，指定了 `dirfd`，且传入的 `path` 本身就不是以 "/" 开头的绝对路径，那么我们就调整这个 `path`，在调用 `ename` 之前，将其补全为绝对路径即可。

```

uint64
sys_open(void)
{
    char path[FAT32_MAX_PATH];
    int fd, omode, dirfd;
    struct file *f;
    struct dirent *ep;

    if (argint(0, &dirfd) < 0 || argstr(1, path, FAT
32_MAX_PATH) < 0 || argint(2, &omode) < 0)
        return -1;

    // printf("sys_open: dirfd=%d, path=%s, omode=%d
\n", dirfd, path, omode);

    if (path[0] == '/' || dirfd == AT_FDCWD)
    { // 绝对路径或者没有指定 dirfd (AT_FDCWD)，直接使用
path 进行查找
        ;
    }
}

```

```

else
{
    struct file *dirf;
    if (argfd(0, 0, &dirf) < 0 || !(dirf->ep->attribute & ATTR_DIRECTORY))
    {
        // 这一步检查 dirfd 是否有效，并且对应一个目录dirf。如果不满足条件，返回 -1。
        return -1;
    }

    struct dirent *curr = dirf->ep;
    char fullpath[FAT32_MAX_PATH];

    // 把指针 p_out 放到数组的最后面，准备从后往前写
    char *p_out = fullpath + FAT32_MAX_PATH - 1;
    *p_out = '\0'; // 字符串结尾符

    // 1. 把用户传进来的相对路径塞到最尾部
    int path_len = strlen(path);
    if (path_len >= FAT32_MAX_PATH)
        return -1;
    p_out -= path_len;
    // 这里用 memmove 只移动字符，不带 \0
    memmove(p_out, path, path_len);

    // 2. 循环向上回溯，不断把父目录名拼接到前面
    while (curr != NULL && curr->parent != curr && curr->parent != NULL)
    {
        int len = strlen(curr->filename);

        if (p_out - fullpath < len + 1)
        {
            return -1; // 路径太长
        }

        // 往前移动指针并写入 '/'

```



```

        p_out--;
        *p_out = '/';

        // 往前移动指针并写入当前目录名
        p_out -= len;
        memmove(p_out, curr->filename, len); // 同样
只移动字符

        curr = curr->parent;
    }

    // 3. 处理根目录的 '/'
    if (p_out > fullpath)
    {
        p_out--;
        *p_out = '/';
    }
    safestrcpy(path, p_out, FAT32_MAX_PATH);
}

    printf("sys_open: full path=%s, omode=%d\n", path, omode);

    ...

}

```

这样以来整个openat的逻辑链就完整了，这时，我们再进行检查，输出就会是这样

```

init: starting openat
===== START test_openat =====
sys_open: full path=./mnt, omode=2097152
open dir fd: 3
sys_open: full path=/mnt/test_openat.txt, omode=66
openat fd: 4
openat success.
===== END test_openat =====

```

顺手，我们把close测试点也直接加进去，一并就都通过了。

▼ fstat:

- 看看需要做什么:

```
#define AT_FDCWD (-100) //相对路径

//Stat *kst;
static struct kstat kst;
void test_fstat() {
    TEST_START(__func__);
    int fd = open("./text.txt", 0);
    int ret = fstat(fd, &kst);
    printf("fstat ret: %d\n", ret);
    assert(ret >= 0);

    printf("fstat: dev: %d, inode: %d, mode: %d, n
link: %d, size: %d, atime: %d, mtime: %d, ctime: %
d\n",
        kst.st_dev, kst.st_ino, kst.st_mode, ks
t.st_nlink, kst.st_size, kst.st_atime_sec, kst.st_
mtime_sec, kst.st_ctime_sec);

    TEST_END(__func__);
}
```

其实是要检查一下test.txt在这个进程中打开时的状态，我们看看kstat是什么结构体

```
struct kstat {
    uint64 st_dev;
    uint64 st_ino;
    mode_t st_mode;
    uint32 st_nlink;
    uint32 st_uid;
    uint32 st_gid;
    uint64 st_rdev;
    unsigned long __pad;
    off_t st_size;
```

```

uint32 st_blksize;
int __pad2;
uint64 st_blocks;
long st_atime_sec;
long st_atime_nsec;
long st_mtime_sec;
long st_mtime_nsec;
long st_ctime_sec;
long st_ctime_nsec;
unsigned __unused[2];
};

```

由于这个系统调用号在内核态的 `sysnum.h` 中已经定义过，我们可以直接查找 `sysfile.c` 中的 `sys_fstat()` 来看看这里的实现情况

- 这是一个很直观的实现，在外面的 `sys_fstat()` 只做套壳，解读 `fd` 为 `f`，并进行 `filestat` 的调用

```

uint64
sys_fstat(void)
{
    struct file *f;
    uint64 st; // user pointer to struct stat

    if (argfd(0, 0, &f) < 0 || argaddr(1, &st) <
    0)
        return -1;
    return filestat(f, st);
}

```

我们直接看 `filestat()` 的实现：

```

// Get metadata about file f.
// addr is a user virtual address, pointing to
// a struct stat.
int
filestat(struct file *f, uint64 addr)
{
    // struct proc *p = myproc();
}

```

```

    struct stat st;

    if(f->type == FD_ENTRY){
        elock(f->ep);
        estat(f->ep, &st);
        eunlock(f->ep);
        // if(copyout(p->pagetable, addr, (char *)&
st, sizeof(st)) < 0)
        if(copyout2(addr, (char *)&st, sizeof(st))
< 0)
            return -1;
        return 0;
    }
    return -1;
}

```

我们直接看 `stat` 这个结构体，按理说，它应该与用户态的 `struct kstat` 完全相同才行

```

struct stat {
    char name[STAT_MAX_NAME + 1];
    int dev;      // File system's disk device
    short type;   // Type of file
    uint64 size;  // Size of file in bytes
};

```

这才是内核态的 `stat`，那么这就完全错误了，当用户态已经在用相当完整的 `kstat`，内核态的 `sys_fstat` 还在返回一个指向小小 `stat` 的指针，这就导致我们运行这个测试样例时，结果全是0。

- 于是我们要定义一个与 `kstat` 完全一样的结构体才可以。注意不要删除原有的 `stat` 结构，因为在 `filestat()` 中，已经实现了向 `stat` 中传递参数的逻辑 `estat`，而 `stat` 中的这几个现有成员，也是 `kstat` 中有的几个成员，我们不需要额外的逻辑去调整这个 `estat` 了。

```

typedef unsigned int mode_t;
typedef long int off_t;

struct kstat

```

```

{
    uint64 st_dev;
    uint64 st_ino;
    mode_t st_mode;
    uint32 st_nlink;
    uint32 st_uid;
    uint32 st_gid;
    uint64 st_rdev;
    unsigned long __pad;
    off_t st_size;
    uint32 st_blksize;
    int __pad2;
    uint64 st_blocks;
    long st_atime_sec;
    long st_atime_nsec;
    long st_mtime_sec;
    long st_mtime_nsec;
    long st_ctime_sec;
    long st_ctime_nsec;
    unsigned __unused[2];
};

```

- 此后，我们在原有的filestat基础上，做一定的调整，使得原本的stat成员可以直接传给更大的kstat结构体

```

int
filestat(struct file *f, uint64 addr)
{
    // struct proc *p = myproc();
    struct stat st;

    if(f->type == FD_ENTRY){
        elock(f->ep);
        estat(f->ep, &st);
        eunlock(f->ep);

        struct kstat kst;
        memset(&kst, 0, sizeof(kst));
    }
}

```

```

    kst.st_dev = st.dev;
    kst.st_size = st.size;

    // if(copyout(p->pagetable, addr, (char *)&
    st, sizeof(st)) < 0)
        if(copyout2(addr, (char *)&kst, sizeof(kst)) < 0)
            return -1;
        return 0;
    }
    return -1;
}

```

但其他的kstat成员我们还是不知道怎么实现，这时候我们不妨看看测试样例的要求：

```

def test(self, data):
    self.assert_ge(len(data), 2)
    res = re.findall("fststat ret: (\d+)", data[0])
    if res:
        self.assert_equal(res[0], "0")
        res = re.findall(r"fststat: dev: \d+, inode: \d+, mode: (\d+), nlink: (\d+), size: \d+, atime: \d+, mtime: \d+, ctime: \d+", data[1])
        if res:
            self.assert_equal(res[0][1], "1")

```

这里虽然要求了输出中这些位置的参数都是大于等于0的整数，但最终第二条判断却只检验了 `nlink` 这一项是否=1，`nlink` 事实上就是硬链接数量

而 `nlink` 代表什么呢：

当前文件系统中，有多少个“文件名（目录项）”共同指向这一个物理实体（`dirent`）。

- 普通文件：当你新建一个普通文件（比如 `touch my.txt`）时，系统上只有一个名字指向这块数据，所以它的 `nlink` 是 1。如果你给它建了一个硬链接，`nlink` 就会变成 2。

- 空目录：假设你新建了一个空目录 `mkdir mydir`，它的 `nlink` 初始就是 2！为什么是 2？因为有两个目录名指向它：父目录里叫作 `mydir` 的那个目录项。它自己里面的 `.`（代表当前目录）这个专属隐藏项。

（如果 `mydir` 里面又建了一个子目录 `sub`，因为 `ubuntu` 也会指向上级目录，那么 `mydir` 的 `nlink` 就会变成 3，也就是多了一个来自 `sub` 的 `..`）

这样，我们就知道应该在这里根据情况不同设置初始的 `nlink`，而在这里，`stat` 还有一个成员 `type` 我们需要用，它是由 `st->type = (de-attribute & ATTR_DIRECTORY) ? T_DIR : T_FILE;` 决定的，这个标明了该文件的类型是目录还是文件，所以我们可以对齐到 `kstat` 中

```
struct kstat kst;
memset(&kst, 0, sizeof(kst));
kst.st_dev = st.dev;
kst.st_size = st.size;
if (st.type == T_DIR)
{
    kst.st_mode = DIR;
    kst.st_nlink = 2; // 目录至少有两个链接：一个来自父目录，另一个来自 . 自身
}
else if (st.type == T_FILE)
{
    kst.st_mode = FILE;
    kst.st_nlink = 1;
}
else
    kst.st_mode = 0;
```

注意这里要在 `file.c` 的开头添加 `#include "include/fcntl.h"` 以便我们使用其中定义的宏 `DIR` 和 `FILE`，这两个宏的定义，又需要在 `fcntl.h` 中进行补充，仿照用户态的定义，有：

```
#define DIR 0x040000
#define FILE 0x100000
```

- 这里我们没有太过关注完全的 `kstat` 定义，只是实现了通过测试点的部分。

▼ vma的管理

至此，我们完成了 `mmap` 的前置任务，可以开始真正实现 `mmap` 了。

而 `mmap`，实际上就是将磁盘上的一个文件（在测试样例中是 `test_mmap.txt`）直接映射到用户态的一个进程的虚拟内存空间中，这样我们在读写文件时，就不必依靠慢吞吞的读写文件函数，而是可以直接在这个进程的虚拟空间中，像读写内存空间一样读写文件的内容，随后，在 `munmap` 时，再把修改写回磁盘

- 首先我们分析一下 `mmap` 调用的参数情况

```
void *mmap(void *start, size_t len, int prot, int
flags, int fd, off_t off)
{
    return syscall(SYS_mmap, start, len, prot, fla
gs, fd, off);
}
int munmap(void *start, size_t len)
{
    return syscall(SYS_munmap, start, len);
}
```

传了6个参数，而如果我们再看一下它下面的 `munmap` 调用，我们发现在取消这个映射时，并没有传入 `fd` 参数，也就是说当用户态告知内核取消映射时，内核必须要自己记忆这个 `array` 的地址本身是归属于哪一个文件、以及何种权限才行。

而在我们目前的 `struct proc` 中，没有这样的成员，可以帮助内核将 `mmap` 传入的这些参数一一记录。所以要去创建一个这样的结构

```
// 仿照用户态的宏定义，确保内核也能看懂这些 flag
// for mmap
#define PROT_NONE 0
#define PROT_READ 1
#define PROT_WRITE 2
#define PROT_EXEC 4
#define PROT_GROWSDOWN 0X01000000
#define PROT_GROWSUP 0X02000000
```



```

#define MAP_FILE 0
#define MAP_SHARED 0x01
#define MAP_PRIVATE 0x02
#define MAP_FAILED ((void *) -1)

struct vma
{
    int valid;          // 槽位是否可用：0代表空闲，1代表正在使用
    uint64 addr;        // 分配给用户的虚拟地址起始点（账本记录的地址）
    int length;         // 映射的长度
    int prot;           // 权限（PROT_READ, PROT_WRITE）
    int flags;          // 标志（MAP_SHARED 等）
    struct file *f;      // 指向被映射的打开文件
    int offset;         // 文件的偏移量
};

// Per-process state
struct proc
{
    ...
    struct vma vmas[NVMA];    // 进程的虚拟内存区域表
};

```

这里的 `NVMA`，我们在 `param.h` 中给出它的定义 `#define NVMA 16`，为什么是这个数，还记得在之前我们看到实现 `openat` 过程中，关注到 `struct proc` 中，打开文件数的上限 `NOFILE` 其实也是16，也就是说最多最多，每个被进程打开的文件都会被 `mmap` 到一个 `vma` 上。

- 接下来，由于在 `proc` 结构体中多出来了这个条目，我们需要在现有的这些调用中，把这部分内容的初始化与回收逻辑进行补充。
 - 这一部分是在找到空闲进程时，应该同时将该进程的 `valid` 位设为0

```

// Look in the process table for an UNUSED pro
C.
// If found, initialize state required to run i

```

```

n the kernel,
// and return with p->lock held.
// If there are no free procs, or a memory allo
cation fails, return 0.
static struct proc *
allocproc(void)
{
    ...
found:
    p->pid = allocpid();
    ...

    for (int i = 0; i < NVMA; i++)
    {
        p->vmass[i].valid = 0;
    }

    p->kstack = VKSTACK;

    ...
    return p;
}

```

- 在初始化做完之后，一个进程的变化还涉及到 `fork`、`exit`、`exec` 这些调用，因此我们也需要一一补充内容

这部分是对 `fork` 的补充，我们需要调用 `filedup()` 来在子进程中增加一次文件的引用，防止父进程死掉以后把子进程的文件也收回了。

```

// Create a new process, copying the parent.
// Sets up child kernel stack to return as if f
rom fork() system call.
int fork(void)
{
    ...

    for(i = 0; i < NVMA; i++) {
        if(p->vmass[i].valid) {
            np->vmass[i] = p->vmass[i];

```

```

        if(np->vmass[i].f) {
            filedup(np->vmass[i].f);
        }
    }
}

np->state = RUNNABLE;

release(&np->lock);

return pid;
}

```

在 `exec` 的过程中，我们到 `exec.c` 中阅读原有的 `exec` 函数逻辑，就可以知道，在 `exec` 要执行一个新的程序时，首先要做的就是将旧页表 `p->kpagetable` 移到一个新的地方去，让现有的 `myproc` 的 `kpagetable` 位置空出来给新的执行程序让位置，所以对应的 `vma` 也需要被清除掉。

```

int exec(char *path, char **argv)
{
    ...
    p->trapframe->sp = sp;           // initial stack pointer

    // Clear VMAs across exec, unmap and writeback if needed
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid)
        {
            // Note: xv6 original does not write back on exec as standard POSIX expects exec to unmap
            if ((p->vmass[i].flags & MAP_SHARED) && (p->vmass[i].prot & PROT_WRITE))
            {
                filewrite(p->vmass[i].f, p->vmass[i].addr, p->vmass[i].length);
            }
            fileclose(p->vmass[i].f);
        }
    }
}

```

```

        p->vmass[i].valid = 0;
    }
}
proc_freepagetable(oldpagetable, oldsz);

...
}

```

- 在exit时，这个进程的所有内容都会被清理掉，而 `vma` 也会被清理，但要想实现懒分配，在exit清理掉 `vma` 对于文件修改的记录以前，我们就需要对这个 `vma` 记录的改动进行真实的回写操作。

什么是直接分配在直接分配中，如果使用 `mmap` 映射了一块物理内存到虚拟地址中，内核会开始使用 `uvmmalloc`，这个包装好的函数则又会调用若干次真实物理内存分配函数 `kalloc`，极力寻找空闲的物理内存页，映射到页表上，并将这些虚拟地址分配给这个 `mmap` 映射的文件，将这个文件中的所有内容读到些页表上去。但实际上，如果用户态需要映射一个1GB大小的文档，却只需要查看开头几KB部分的内容并修改，这种直接分配的办法就会显得很不妥当。

那什么是懒分配：而在懒分配中，`vma` 只负责记录，在第一次调用 `mmap` 时，内核不会分配内存给这个映射的文件，也不会读这个文件的内容，只是登记：这一块虚拟内存的地址空间已经被这个文件占用了。直到用户第一次对这个文件实行读写操作时，会用之前登记到的虚拟地址来访问真实的物理空间，但由于此前这个位置根本没有被分配物理空间，用虚拟地址对应的valid位根本就是0，于是触发缺页中断，但内核只会为其 `kalloc` 一页（4KB）的空间，挂到页表上然后将valid位设为1，之后程序继续运行。

- exit时，进程的 `p->state` 会变成 `ZOMBIE`，由父进程的 `wait` 等待，并执行 `freeproc` 的彻底清理函数。

```

int wait(uint64 addr, int wait_pid)
{
    ...
    if (np->state == ZOMBIE)
    {
        // Found one.
        pid = np->pid;
        int status = np->xstate << 8;
    }
}

```

```

        if (addr != 0 && copyout2(addr, (char
*)&status, sizeof(status)) < 0)
        {
            release(&np->lock);
            release(&p->lock);
            return -1;
        }
        freeproc(np);    // <-就是这个地方调用了
freeproc()
        release(&np->lock);
        release(&p->lock);
        return pid;
    }
    ...
}

```

而 `freeproc()` 会调用更底层一点的 `proc_freepagetable(p->pagetable, p->sz)`，我们查看这个函数的定义，会发现它实际上调用了2次 `vmunmap` 分别解除了跳板（`TRAMPOLINE`）和陷阱帧（`TRAPFRAME`）的映射，而查看 `uvmfree`，我们发现它实际上也是在调用 `vmunmap(pagetable, 0, PGROUNDUP(sz) / PGSIZE, 1)`；解除从0开始到堆顶sz这部分的映射后，才进行最底层的 `kfree`。这说明如果我们希望释放进程，就必须调用 `vmunmap` 将其中页表的映射关系进行解除。

```

void proc_freepagetable(pagetable_t pagetable,
uint64 sz)
{
    vmunmap(pagetable, TRAMPOLINE, 1, 0);
    vmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmfree(pagetable, sz);
}

```

- 。为了满足懒分配，在正式 `exit` 之前，进程的 `mmap` 区域（即 `vma` 账本的位置）会出现很多空洞（虽然分配了虚拟内存地址，但未分配真实物理内存页），而在现有的 `vmunmap` 函数中，如果有这样大量的缺页情况，它会一直爆出 `panic`，所以为了补充懒分配的漏洞，我们要调整一下 `vmunmap` 的逻辑。

```

// Remove npages of mappings starting from va.
va must be
// page-aligned. The mappings must exist.
// Optionally free the physical memory.
// 这个函数的作用就是检查页表中的每一个pte（页表项）是
// 否可以被
void vmunmap(pagetable_t pagetable, uint64 va,
uint64 npages, int do_free)
{
    uint64 a;
    pte_t *pte;

    if ((va % PGSIZE) != 0)
        panic("vmunmap: not aligned");

    for (a = va; a < va + npages * PGSIZE; a += P
GSIZE)
    {
        if ((pte = walk(pagetable, a, 0)) == 0)
            // panic("vmunmap: walk");
            continue; // 没有pte, 说明根本没有分配物理内
存, 直接继续下一个地址
        if ((*pte & PTE_V) == 0)
            // panic("vmunmap: not mapped");
            continue; // 没有映射, 说明根本没有分配物理内
存, 直接继续下一个地址
        if (PTE_FLAGS(*pte) == PTE_V)
            panic("vmunmap: not a leaf");
        if (do_free)
        {
            uint64 pa = PTE2PA(*pte);
            kfree((void *)pa);
        }
        *pte = 0;
    }
}

```

- 接下来，`exit` 之前，还需要把所有记录了dirty的 `vma` 进行真正物理内存写回的逻辑。`write`其实已经被实现过了，而它的最终逻辑是调用底层的 `filewrite`

```
uint64
sys_write(void)
{
    struct file *f;
    int n;
    uint64 p;

    if (argfd(0, 0, &f) < 0 || argint(2, &n) < 0
    || argaddr(1, &p) < 0)
        return -1;

    return filewrite(f, p, n);
}
```

我们查看这个filewrite是否可以直接被用来做我们的exit前vma写回

```
// Write to file f.
// addr is a user virtual address.
int filewrite(struct file *f, uint64 addr, int
n)
{
    int ret = 0;
    ... // 我们在这里只用关注写回的file类型为entry（即文
件条目）时的情况
    else if (f->type == FD_ENTRY)
    {
        elock(f->ep);
        if (ewrite(f->ep, 1, addr, f->off, n) == n)
        {
            ret = n;
            f->off += n;
        }
        else
        {

```

```

        ret = -1;
    }
    eunlock(f->ep);
}
else
{
    panic("filewrite");
}

return ret;
}

```

而这里的 `ewrite` 才是最终的写回核心函数，而在 `ewrite` 中，内核试图将 `addr` 开始的 `n` 大小的空间中进行写回，但也许在这些空间中存在由于懒分配导致的缺页，这就会有问题。

所以我们还必须得像 `vmunmap` 那样一页一页查询缺页情况，过滤掉未分配的物理页。我们最好自己实现一个查询与写回的函数。

我们不能调用 `vmunmap` 中那个逐页查询缺页情况的 `walk` 函数，这个函数只会返回一个 `pte` 指针，但我们需要快速获得一个虚拟地址 `va` 对应的物理地址 `pa`，用来查看这个物理页是否已经被分配出去了，所以这里我们选择使用 `walkaddr`。注意：要记得在 `proc.h` 中声明这个函数。

```

// 将 VMA 指定范围内的共有且可写数据写回文件
void vma_writeback(pagetable_t pagetable, uint64
addr, uint64 length, struct vma *v)
{
    if ((v->flags & MAP_SHARED) && (v->prot & PROT_WRITE)) // 只有共享映射且具有写权限的内容需要写回底层文件
    {
        for (uint64 va = addr; va < addr + length;
va += PGSIZE) // 从vma账本上分配给用户的虚拟地址起点开始
        { // 逐页进行查询
            uint64 pa = walkaddr(pagetable, va);
            if (pa != 0) // 如果这个物理页确实存在
            {
                uint64 n = PGSIZE;

```



```

        if (va + n > addr + length) // 检查这个
            地址加上新页大小是否超过了给分配出的地址
            n = (addr + length) - va;

        elock(v->f->ep);
        ewrite(v->f->ep, 1, va, v->offset + (va
- v->addr), n); // 真正的ewrite
        eunlock(v->f->ep);
    }
}
}
}

```

- 接下来我们在exit中进行这一步的修改，这里与A神的博客存在不一样的地方，在解除映射时，没有检测flags是否为SHARED，这里Gemini的回应是：

无论是 MAP_SHARED 还是 MAP_PRIVATE 或者匿名映射（匿名映射可能都没背后的文件）：只要是进程通过 mmap 分配出的物理内存页面，在它 exit (或者 exec) 的时候，都必须把物理内存释放掉 (do_free = 1)，交还给内核的空闲内存池。

MAP_SHARED 只是说，释放前这块数据需要被 writeback 写回到底层的文件而已（这步正是 vma_writeback 做的）。而 MAP_PRIVATE 的数据只是被私有修改，死的时候直接扔掉（无需写回），但底层的物理 RAM 还给内核这个事实是毫无区别的。

```

void exit(int status)
{
    struct proc *p = myproc();

    if (p == initproc)
        panic("init exiting");

    // Close all open files.
    for (int fd = 0; fd < NOFILE; fd++)
    {
        if (p->ofile[fd])
        {
            struct file *f = p->ofile[fd];

```

```

        fclose(f);
        p->ofile[fd] = 0;
    }
}

// 清理 VMA 并将共享映射的数据写回文件
for (int i = 0; i < NVMA; i++)
{
    if (p->vmass[i].valid)
    {
        vma_writeback(p->pagetable, p->vmass[i].addr, p->vmass[i].length, &p->vmass[i]);
        // 这里要记着, 把这个vma条目映射的物理空间解绑
        vmunmap(p->pagetable, p->vmass[i].addr, PG
ROUNDUP(p->vmass[i].length) / PGSIZE, 1);
        if (p->vmass[i].f)
        {
            // 如果这个vma映射的是一个文件, 就关闭它
            fclose(p->vmass[i].f);
            p->vmass[i].f = NULL;
        }
        p->vmass[i].valid = 0;    // 标记为未使用
    }
}

eput(p->cwd);
p->cwd = 0;

...

}

```

- 自此 `exit` 的逻辑我们补充完整了, 但在 `exec` 中, 我们其实还需要再做一些调整, 因为不论是 `exit` 的“完全抹除”还是 `exec` 的“夺舍转生”, 旧的内容都会完全被摧毁, 那么在 `exec` 时, 我们其实也需要将 `mmap` 的映射内容进行写回的判断。

```

int exec(char *path, char **argv)
{
    ...
    p->trapframe->sp = sp;          // initial stack
    pointer                          // Clear VMAs across
    exec, unmap and writeback if needed
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid)
        {
            // Note: xv6 original does not write back on
            exec as standard POSIX expects exec to unmap
            vma_writeback(oldpagetable, p->vmass[i].addr,
            p->vmass[i].length, &p->vmass[i]);
            vmunmap(oldpagetable, p->vmass[i].addr, PGR0U
            NDUP(p->vmass[i].length) / PGSIZE, 1);
            if (p->vmass[i].f)
            {
                fclose(p->vmass[i].f);
                p->vmass[i].f = NULL;
            }
            p->vmass[i].valid = 0;
        }
    }
    proc_freepagetable(oldpagetable, oldsz);
    ...
}

```

- 现在我们终于可以对mmap动手了

```

uint64
sys_mmap(void)
{
    uint64 addr, length, offset;
    int prot, flags, fd;

    if (argaddr(0, &addr) < 0 || argaddr(1, &length) <

```

```

0 ||
    argint(2, &prot) < 0 || argint(3, &flags) < 0 |
|
    argint(4, &fd) < 0 || argaddr(5, &offset) < 0)
{
    return -1;
}

struct proc *p = myproc();
struct file *f = NULL;

// fd描述符在范围内且进程的ofile表中存在这个文件的指针
if (fd >= 0 && fd < NOFILE && p->ofile[fd])
{
    f = p->ofile[fd];
}
else
{
    // fd < 0 (通常为 -1) 可能是匿名映射，但如果存在文件映射
    则需要合法的 fd
    if (fd != -1)
        return -1;
}

if (f)
{
    if ((prot & PROT_READ) && !f->readable) // 用户要
    求读文件但这个文件不可读
        return -1;
    if ((prot & PROT_WRITE) && (flags & MAP_SHARED) &
    & !f->writable) // 用户要求写这个文件，并且不是私有的映射，
    但文件不可写
        return -1;
}

// 找到该进程中一个可用的vma项
struct vma *v = NULL;
for (int i = 0; i < NVMA; i++)

```

```

{
    if (p->vmass[i].valid == 0)
    {
        v = &p->vmass[i];
        break;
    }
}

if (v == NULL)
    return -1;

if (addr == 0)
{
    // 向下寻找空闲的虚拟地址空间
    addr = TRAPFRAME;
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid && p->vmass[i].addr < addr)
            {// 这里的逻辑是：跳过之前的所有在该进程空间中已分配的v
ma, 找到新的需要分配的vma地址的天花板
                addr = p->vmass[i].addr;
            }
    }
    // 向下分配并且对齐
    addr -= PGROUNDUP(length);
    addr &= ~(PGSIZE - 1);
}

// 检查现在选定的新的mmap块起始地址是否低于目前的堆顶地址p-
>sz
if (addr < PGROUNDUP(p->sz))
{
    return -1;
}

v->valid = 1;
v->addr = addr;
v->length = length;

```

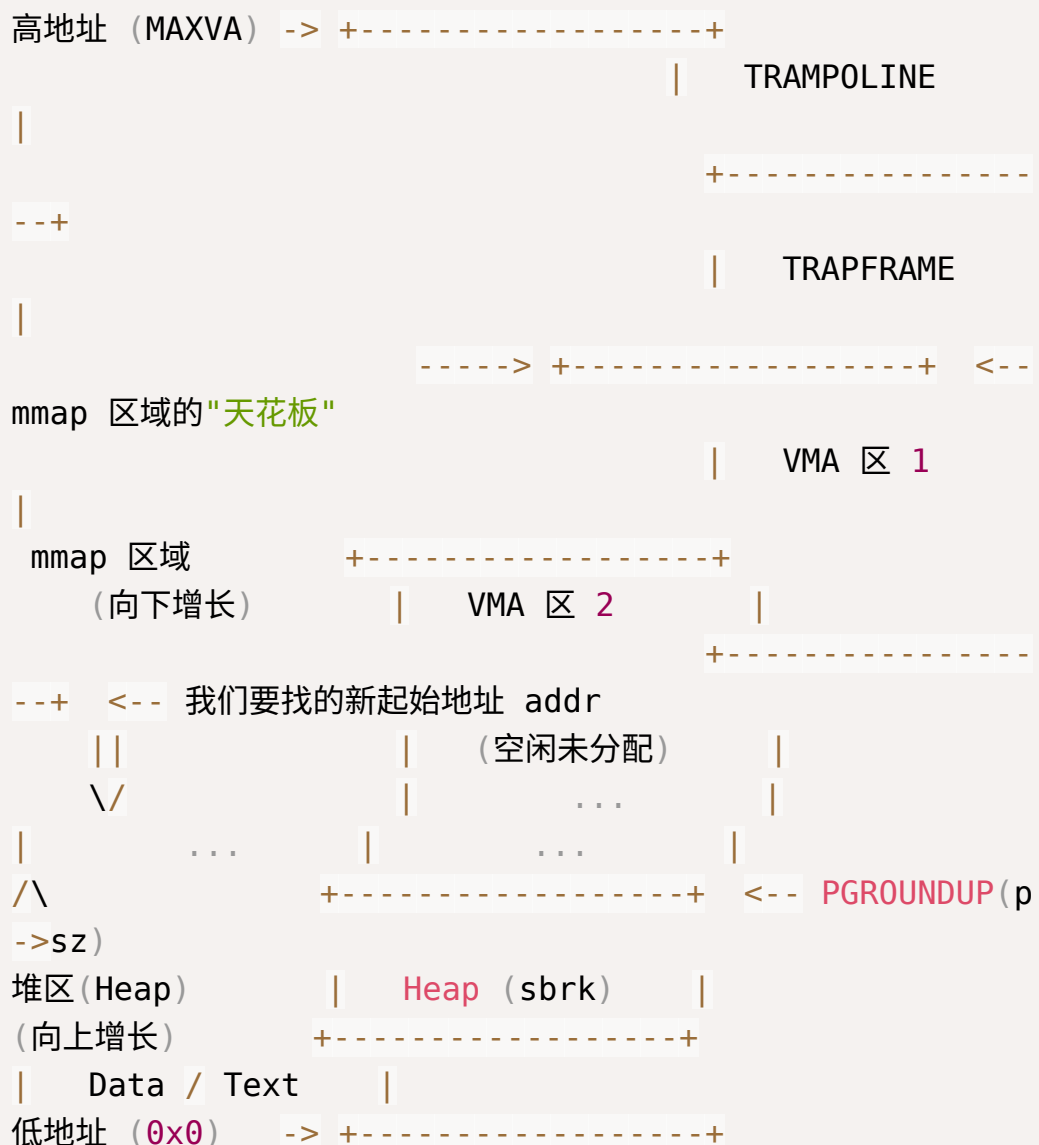
```

v->prot = prot;
v->flags = flags;
v->offset = offset;
v->f = f ? filedup(f) : NULL; // 这里一定要注意: mmap
映射的file, 其引用数必须增加

return v->addr;
}

```

在这里, 有一个宏小工具 `PGROUNDUP`, 它用来进行空间取整, 由于内存映射的最小单位是页 (4096Byte), 所以我们要对用户申请的 `length` 进行页取整。下面这个是一个进程的虚拟内存空间的简图



这样，我们就完成了 `mmap` 中对于 `vma` 这个新逻辑的注册行为。

- 如果在这一步我们就直接测试，应该会得到这样的报错。

```
===== START test_mmap =====
file len: 27
mmap content:
usertrap(): unexpected scause 0x000000000000000d pid=
31 mmap
sepc=0x00000000000001a0 stval=0x0000003fffffd000
init: starting munmap
===== START test_munmap =====
```

这是因为用户态执行到直接读取 `array` 内存这一步时，我们的 `mmap` 虽然给它注册了位置，但并没有真正把物理页给它映射过去，所以内核抛出 `scause 0xd` 缺页异常，因此我们需要正视处理一下由于 `mmap` 引发的缺页异常，让它能够分配物理页给映射，而不是直接抛出异常。

来到 `trap.c` 中，我们对 `usertrap` 这个函数进行修改

```
else if ((which_dev = devintr()) != 0)
{
    // ok
}
else if (r_scause() == 13 || r_scause() == 15 || r_
scause() == 12)
{ // Load / Store / Instruction Page Fault
    uint64 va = r_stval();
    struct vma *v = NULL;
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid && va >= p->vmass[i].addr &
& va < p->vmass[i].addr + p->vmass[i].length)
        {
            v = &p->vmass[i];
            break;
        }
    }

    if (v != NULL)
```

```

{
    uint64 va_align = PGROUNDOWN(va);
    char *mem = kalloc();
    if (mem == 0)
    {
        p->killed = 1;
    }
    else
    {
        memset(mem, 0, PGSIZE);
        if (v->f)
        {
            // 如果是从文件映射，现在把它读进来
            uint64 offset = v->offset + (va_align - v->
addr);
            uint64 n = PGSIZE;
            if (va_align + PGSIZE > v->addr + v->length)
            {
                n = v->addr + v->length - va_align;
            }

            elock(v->f->ep);
            eread(v->f->ep, 0, (uint64)mem, offset, n);
            eunlock(v->f->ep);
        }

        int prot = PTE_U | PTE_V;
        int kprot = PTE_V;
        if (v->prot & PROT_READ)
        {
            prot |= PTE_R;
            kprot |= PTE_R;
        }
        if (v->prot & PROT_WRITE)
        {
            prot |= PTE_W;
            kprot |= PTE_W;
        }
        if (v->prot & PROT_EXEC)

```



```

    {
        prot |= PTE_X;
        kprot |= PTE_X;
    }

    // 映射到用户页表
    if (mappages(p->pagetable, va_align, PGSIZE,
        (uint64)mem, prot) != 0)
    {
        kfree(mem);
        p->killed = 1;
    }
    // k210 需要同步映射到进程对应的内核页表 kpagetabl
e
    else if (mappages(p->kpagetable, va_align, PG
SIZE, (uint64)mem, kprot) != 0)
    {
        vmunmap(p->pagetable, va_align, 1, 1);
        p->killed = 1;
    }
}
}
else
{
    // 如果没找到 vma, 说明是真的越界访问, 杀死进程
    printf("\nusertrap(): page fault but vma not fo
und! va=%p\n", r_stval());
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmas[i].valid)
        {
            printf("  vma[%d]: addr=%p, length=%d\n",
i, p->vmas[i].addr, p->vmas[i].length);
        }
    }
    printf("\nusertrap(): unexpected scause %p pid
=%d %s\n", r_scause(), p->pid, p->name);
    printf("          sepc=%p stval=%p\n", r_sepc

```

```

(), r_stval());
    p->killed = 1;
}
}

```

- 但只改变这里的缺页异常处理函数，再进行测试，仍然得不到测试样例要求的
mmap content: Hello, mmap successfully!，不过我们的 usertrap 与 mmap 都已经正常实现了，这里就是打印时的问题了。这是因为原有的copy函数检测到如果地址大于堆顶，就直接拒绝copy，但我们分配的 mmap 块却远远高于这个地址，所以我们需要修改一下这几个函数的逻辑。

```

int copyout2(uint64 dstva, char *src, uint64 len)
{
    uint64 sz = myproc()->sz;
    if (dstva + len > sz || dstva >= sz)
    {
        // 检查这个拷贝的地址是否在某个有效的vma地址内
        int in_vma = 0;
        struct proc *p = myproc();
        for (int i = 0; i < NVMA; i++)
        {
            if (p->vmass[i].valid && dstva >= p->vmass[i].addr
                && dstva < p->vmass[i].addr + p->vmass[i].length)
            {
                in_vma = 1;
                break;
            }
        }
        if (!in_vma)
            return -1;
    }
    memmove((void *)dstva, src, len);
    return 0;
}

int copyin2(char *dst, uint64 srcva, uint64 len)
{
    uint64 sz = myproc()->sz;

```

```

if (srcva + len > sz || srcva >= sz)
{
    int in_vma = 0;
    struct proc *p = myproc();
    for (int i = 0; i < NVMA; i++)
    {
        if (p->vmass[i].valid && srcva >= p->vmass[i].addr
            && srcva < p->vmass[i].addr + p->vmass[i].length)
        {
            in_vma = 1;
            break;
        }
    }
    if (!in_vma)
        return -1;
}
memmove(dst, (void *)srcva, len);
return 0;
}

int copyinstr2(char *dst, uint64 srcva, uint64 max)
{
    int got_null = 0;
    uint64 sz = myproc()->sz;
    struct proc *proc = myproc();

    while (max > 0)
    {
        if (srcva >= sz)
        {
            int in_vma = 0;
            for (int i = 0; i < NVMA; i++)
            {
                if (proc->vmass[i].valid && srcva >= proc->vmass[i].addr
                    && srcva < proc->vmass[i].addr + proc->vmass[i].length)
                {
                    in_vma = 1;

```

```

        break;
    }
}
if (!in_vma)
    break;
}

char *p = (char *)srcva;
if (*p == '\0')
{
    *dst = '\0';
    got_null = 1;
    break;
}
else
{
    *dst = *p;
}
--max;
srcva++;
dst++;
}
if (got_null)
{
    return 0;
}
else
{
    return -1;
}
}

```

这样，如果这个拷贝的地址确实在某个有效的 `vma` 地址内，那就可以正常进行拷贝了。

- 自此，mmap的工作告一段落。

▼ munmap:

- 看看需要做什么：

```

void test_munmap(void){
    TEST_START(__func__);
    char *array;
    const char *str = "  Hello, mmap successfully!";
    int fd;

    fd = open("test_mmap.txt", O_RDWR | O_CREATE);
    write(fd, str, strlen(str));
    fstat(fd, &kst);
    printf("file len: %d\n", kst.st_size);
    array = mmap(NULL, kst.st_size, PROT_WRITE | PROT
_READ, MAP_FILE | MAP_SHARED, fd, 0);
    //printf("return array: %x\n", array);

    if (array == MAP_FAILED) {
        printf("mmap error.\n");
    }else{
        //printf("mmap content: %s\n", array);

        int ret = munmap(array, kst.st_size);
        printf("munmap return: %d\n",ret);
        assert(ret == 0);

        if (ret == 0)
            printf("munmap successfully!\n");
        }
        close(fd);

        TEST_END(__func__);
    }
}

```

`munmap` 负责将 `mmap` 的映射全部解除，并且在解除之前，看看是否要写回物理空间。其实我们在这里不需要过多关注测试样例，只要按照 `mmap` 的逻辑，将 `munmap` 实现回收的功能即可。这里就不再过多分析了。

```

uint64
sys_munmap(void)
{

```

```

uint64 addr, length;
if (argaddr(0, &addr) < 0 || argaddr(1, &length) <
0)
{
    return -1;
}

struct proc *p = myproc();
struct vma *v = NULL;

for (int i = 0; i < NVMA; i++)
{
    // 允许部分或完整 unmap, 只要 addr 匹配就行
    if (p->vmas[i].valid && addr >= p->vmas[i].addr &
& addr < p->vmas[i].addr + p->vmas[i].length)
    {
        v = &p->vmas[i];
        break;
    }
}

if (v == NULL)
    return -1;

if (v->flags & MAP_SHARED)
{
    vma_writeback(p->pagetable, addr, length, v);
}

// 这里调用 vmunmap 会将我们之前按需分配好的物理页面彻底还
给内核
vmunmap(p->pagetable, addr, PGROUNDUP(length) / PGS
IZE, 1);

if (addr == v->addr && length == v->length)
{
    // 全退还
    if (v->f)

```

```

    {
        fclose(v->f);
        v->f = NULL;
    }
    v->valid = 0;
}
else
{
    // 处理 OSComp 中可能出现的部分截断 munmap (例如头部的
    partial unmap)
    // 根据需要做更精确的设计, 但最简单的是整体释放, 这里只做
    简单的偏移
    v->addr = addr + length;
    v->length -= length;
    v->offset += length;
}

return 0;
}

```